positioning, chains

LightGCN: Mạng tích chập Đồ thị cho Hệ thống Gợi ý

Thuyết trình tiến độ nghiên cứu

Trần Nguyễn Minh Quân

Ngày 25 tháng 10 năm 2026

Trường Đại học Công nghiệp TP.HCM

1. Lý do áp dụng Lý thuyết Đồ thị vào GCN & LightGCN

1.1 Tại sao áp dụng Lý thuyết Đồ thị vào Hệ thống Gợi ý?

Hạn chế của mô hình truyền thống (CNN / Matrix Factorization)

- Dữ liệu tương tác giữa người dùng và sản phẩm mang bản chất phi Euclidean (Non-Euclidean).
- Các mô hình CNN truyền thống chỉ hoạt động tốt trên dữ liệu cấu trúc lưới đều đặn (như hình ảnh), khó biểu diễn trực tiếp các mối quan hệ mạng lưới phức tạp giữa User và Item.

Ưu thế của Lý thuyết Đồ thị & Graph Convolution Networks (GCN)

- Mô hình hóa tự nhiên lịch sử tương tác thành cấu trúc đồ thị (User-Item Interaction Graph).
- Khai thác mối quan hệ xóm giềng bậc cao (High-order connectivity) để học các vector biểu diễn (embeddings) chất lượng hơn so với chỉ dùng thông tin 1-hop.

1.2 Từ GCN chuẩn đến sự đơn giản hóa của LightGCN

Thách thức của GCN chuẩn (như NGCF) trong Lọc cộng tác

- GCN truyền thống được thiết kế cho bài toán phân loại nút có sẵn đặc trưng ngữ nghĩa phong phú (văn bản, thuộc tính).
- Trong Lọc cộng tác (CF), các nút chỉ là các mã ID đơn thuần không chứa ngữ nghĩa sẵn có [3, 4].
- Phép biến đổi đặc trưng (ma trận trọng số W) và hàm kích hoạt phi tuyến (σ) làm tăng độ phức tạp, gây khó khăn cho việc huấn luyện và làm giảm hiệu năng gợi ý.

Sự ra đời của LightGCN

- Tối giản hóa mô hình bằng cách loại bỏ hoàn toàn ma trận trọng số W và hàm kích hoạt phi tuyến σ.
- Chỉ giữ lại phép tích tụ xóm giềng (Neighborhood Aggregation) cốt lõi để làm mịn embedding trên đồ thị.

2. Tổ chức Dữ liệu & Thuật toán LightGCN

2.1 Ma trận tương tác: Ma trận kề khối và Đồ thị lưỡng phân

Biểu diễn Dữ liệu Tương tác dưới dạng Đồ thị

- Giả sử hệ thống có M người dùng (Users) và N sản phẩm (Items).
- Mối quan hệ tương tác được ghi nhận qua **Ma trận tương tác** $R \in \mathbb{R}^{M \times N}$:

$$R_{ui} = \begin{cases} 1, & \text{nếu User } u \text{ đã tương tác với Item } i \\ 0, & \text{ngược lại} \end{cases}$$

- Trong biểu diễn đồ thị: Các nút (nodes) chính là các User và Item; các đường đi/cạnh nối (edges) thể hiện việc người dùng đã từng tương tác với sản phẩm đó.

Đặc tính thực tế & Đồ thị Lưỡng phân (Bipartite Graph)

- Trong thực tế, các User không trực tiếp kết nối với nhau và các Item cũng không trực tiếp kết nối với nhau [5].
- Do đó, tập hợp các nút được chia thành 2 tập hợp hoàn toàn tách biệt. Đồ thị lưỡng phân (đồ thị phân đôi) được áp dụng dựa vào tính chất này.

Ví dụ về mô hình hóa tương tác bằng Đồ thị phân đôi

Tập hợp đỉnh:

- 5 Users: $\mathcal{U} = \{u_1, \ldots, u_5\}$
- 10 Items: $\mathcal{I} = \{i_1, \ldots, i_{10}\}$
- Tổng số nút: $|\mathcal{V}| = 5 + 10 = 15$

Ma trận tương tác

$$R = \begin{pmatrix} 1 & 1 & 0 & \ldots & 0 \\ 0 & 1 & 1 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & 1 \end{pmatrix}$$

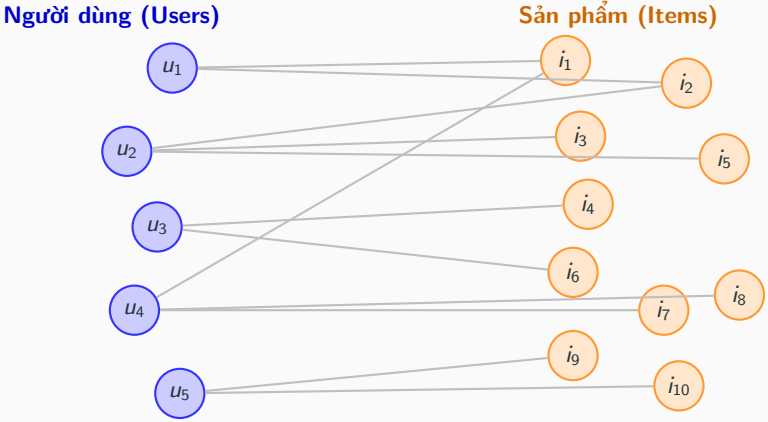
- Với m $= 5$ (dòng) tương ứng 5 user
- Với n $= 10$ (cột) tương ứng 10 item

- Đưa ma trận tương tác R và ma trận chuyển vị $R^\top$ vào ma trận kề toàn cục $A \in \mathbb{R}^{(M+N)\times(M+N)}$:

$$A = \begin{pmatrix} 0_{M\times M} & R_{M\times N} \\ R^\top_{N\times M} & 0_{N\times N} \end{pmatrix}$$

- Khối đường chéo chính là các ma trận 0 ($0_{M\times M}$ và $0_{N\times N}$) phản ánh đúng đặc tính không có tương tác nội bộ giữa User-User hay Item-Item .

2.1.1 Đồ thị phân đôi biểu diễn tương tác



Ví dụ về Đồ thị phân đôi mô tả tương tác giữa 5 Users và 10 Items.

2.2 Kiến trúc Biểu diễn Đa tầng trong LightGCN

Cấu tạo của 1 mạng LGCN gồm các embedding layer như bên dưới:

$$E^{(0)} = \begin{pmatrix} (e_{u_1}^{(0)})^T \\ \vdots \\ (e_{u_M}^{(0)})^T \\ (e_{i_1}^{(0)})^T \\ \vdots \\ (e_{i_N}^{(0)})^T \end{pmatrix} \in \mathbb{R}^{(M+N)\times T}, \quad E^{(1)} = \begin{pmatrix} (e_{u_1}^{(1)})^T \\ \vdots \\ (e_{u_M}^{(1)})^T \\ (e_{i_1}^{(1)})^T \\ \vdots \\ (e_{i_N}^{(1)})^T \end{pmatrix} \in \mathbb{R}^{(M+N)\times T}, \quad \ldots, \quad E^{(K)} = \begin{pmatrix} (e_{u_1}^{(K)})^T \\ \vdots \\ (e_{u_M}^{(K)})^T \\ (e_{i_1}^{(K)})^T \\ \vdots \\ (e_{i_N}^{(K)})^T \end{pmatrix} \in \mathbb{R}^{(M+N)\times T}$$

- Mỗi hàng $(e_{u_m}^{(k)})^T$ hoặc $(e_{i_n}^{(k)})^T$ tương ứng là chuyển vị của một vector biểu diễn riêng lẻ $e_{u_m}^{(k)}, e_{i_n}^{(k)} \in \mathbb{R}^T$ đại diện cho người dùng u_m hoặc sản phẩm i_n tại lớp thứ k.
- $E^{(0)}$: Ma trận embedding khởi tạo (bậc 0), là tham số duy nhất huấn luyện được của mô hình.
- $E^{(1)}, \ldots, E^{(K)}$: Các biểu diễn embedding thu được sau các bước lan truyền đồ thị tuyến tính từ bậc 1 đến bậc K (không bổ sung tham số huấn luyện).
- T: Kích thước số chiều của không gian embedding (embedding dimension).

Tại mỗi lớp lan truyền k + 1, vector biểu diễn của người dùng u và sản phẩm i được cập nhật thuần túy qua phép lấy tổng có trọng số từ các nút liền kề trong đồ thị (neighborhood aggregation):

$$e_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u| \cdot |\mathcal{N}_i|}} e_i^{(k)}$$

$$e_i^{(k+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i| \cdot |\mathcal{N}_u|}} e_u^{(k)}$$

- $\mathcal{N}_u$ là tập hợp các item mà người dùng u đã tương tác.
- $\mathcal{N}_i$ là tập hợp các người dùng đã tương tác với item i.

Tuy nhiên, để thuận tiện trong việc tính toán lan truyền bằng Pytorch, ta sẽ sử dụng dạng ma trận của các phép tính trên.

Ma trận kề chuẩn hóa đối xứng và phép lan truyền toàn cục

Đặt $D \in \mathbb{R}^{(M+N)\times(M+N)}$ là ma trận bậc đường chéo của A, với $D_{tt} = \sum_s A_{ts}$ hay nói cách khác, các phần tử trên đường chéo chính của D là số lượng các item mà user u đã tương tác và ngược lại. Ma trận kề chuẩn hóa đối xứng được xác định:

$$\tilde{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \tag{1}$$

Khi đó, phép lan truyền trên toàn bộ ma trận tại lớp $k+1$ được biểu diễn gọn gàng:

$$E^{(k+1)} = \tilde{A} E^{(k)} = \tilde{A}^{k+1} E^{(0)} \tag{2}$$

Sau khi lan truyền qua K lớp, mô hình thu được $K+1$ biểu diễn embedding cho mỗi nút e_u, e_i. LightGCN tổng hợp các biểu diễn này thông qua tổ hợp tuyến tính:

$$e_u = \sum_{k=0}^{K} \alpha_k e_u^{(k)}, \quad e_i = \sum_{k=0}^{K} \alpha_k e_i^{(k)}$$

- với $\alpha_k = \frac{1}{K+1}$ được cài đặt mặc định, có thể được xem như là 1 tham số của mô hình và tối ưu 1 cách tự động nhưng $\frac{1}{K+1}$ là giá trị tối ưu sau nhiều lần thử nghiệm của tác giả bài báo và không nên tinh chỉnh nhiều nhằm tránh việc mô hình trở nên phức tạp.

Biểu diễn cuối cùng thông qua Layer Combination

$$E = \sum_{k=0}^{K} \alpha_k E^{(k)} = \sum_{k=0}^{K} \alpha_k \tilde{A}^k E^{(0)}$$

Thông thường, số lớp thực nghiệm lý tưởng là $K = 3$ hoặc 4 Điểm số dự đoán mức độ ưa thích giữa người dùng u và item i được tính: $\hat{y_{ui}} = e_u^T e_i$

2.3 Huấn luyện mô hình bằng BPR Loss

Cuối cùng, ta sử dụng hàm loss BPR (Bayesian Personalized Ranking) để huấn luyện mô hình:

$$\mathcal{L}_{\text{BPR}} = -\sum_{u=1}^{M} \sum_{i \in \mathcal{N}_u} \sum_{j \notin \mathcal{N}_u} \ln \sigma \left(\hat{y}_{ui} - \hat{y}_{uj} \right) + \lambda \| E^{(0)} \|^2 \tag{3}$$

- với các j ở đây là các item không tương tác với user u.
- $\hat{y_{ui}}$ là điểm dự đoán cho các item không tương tác j với user u.
- $\lambda \| E^{(0)} \|^2$ là regularization term với λ là hệ số phạt, $E^{(0)}$ là embedding layer ban đầu (tham số duy nhất có thể train được của mô hình).

Để cập nhật tham số, ta sử dụng thuật toán Adam:

$$\mathbf{m}_{t+1} \leftarrow \frac{\partial \mathcal{L}_{\text{BPR}}}{\partial E^{(0)}}\bigg|_{E_t^{(0)}}$$
$$\mathbf{v}_{t+1} \leftarrow \left(\frac{\partial \mathcal{L}_{\text{BPR}}}{\partial E^{(0)}}\bigg|_{E_t^{(0)}}\right)^2.$$

Sau đó áp dụng quy tắc cập nhật nghiệm:

$$E_{t+1}^{(0)} \leftarrow E_t^{(0)} - \alpha \cdot \frac{\mathbf{m}_{t+1}}{\sqrt{\mathbf{v}_{t+1}} + \epsilon},$$

- với $\alpha > 0$ là tốc độ học.

3. Thực nghiệm mô hình